

Progetto Di Reti Logiche

A.A. 2025–2026

Relazione di
Riccardo Mattia & Luca Messenio

Studenti:

10893716

10912026

Docente di riferimento:

Fabio Salice

Indice

1	Introduzione	2
2	Architettura	2
2.1	Porte di Interfaccia	2
2.2	Segnali Interni — Registri (Flip-Flop)	2
2.3	Moduli e processi	3
2.3.1	Processo <code>state_reg</code>	3
2.3.2	Processo <code>datapath_regs</code>	3
2.3.3	Processo <code>comb</code>	3
2.4	Diagramma degli Stati	5
3	Scelte progettuali	6
3.1	Gestione della RAM sincrona write-first	6
3.2	Controllo della lista vuota in <code>S_DECODE</code>	6
3.3	Utilizzo dello stato <code>S_WAIT_DONE</code>	6
3.4	Implementazione operazioni <code>i_op</code>	6
4	Risultati sperimentali	6
4.1	Simulazione Funzionale	6
4.2	Sintesi e Occupazione di Area	7
4.3	Analisi Temporale (Timing)	8
4.4	Tabella riassuntiva	9
4.5	Nota sui dati	9
5	Testbench Ad Hoc	9
5.1	Introduzione alla Strategia di Test	9
5.2	Descrizione dei Testbench per Casi Limite	9
6	Conclusioni	10
6.1	Considerazioni finali sul lavoro svolto	10

1 Introduzione

L'obiettivo del progetto è realizzare in VHDL un componente che gestisce una lista ordinata di task memorizzata su una RAM esterna.

Ogni task è codificato su 8 bit con i primi 6 bit più significativi che rappresentano ID_TASK (6 bit) e gli ultimi 2 bit che rappresentano PRIORITY (2 bit), dove la priorità 0 è la massima e 3 la minima. La RAM è organizzata con l'indirizzo 0 che contiene il numero di task presenti (num_tasks), mentre gli indirizzi 1..N contengono i task ordinati per priorità crescente.

Il modulo implementato supporta quattro operazioni, selezionate dal segnale i_op:

- **00 – Decremento Priorità:** incrementa il valore numerico di priorità di tutti i task con limite a 3 (priorità minima).
- **01 – Estrazione Primo Task:** rimuove il task in testa alla lista, shifta gli altri verso sinistra e restituisce l'ID estratto.
- **10 – Inserimento Task:** inserisce un nuovo task mantenendo l'ordinamento per priorità, mettendolo in coda rispetto a task di pari priorità.
- **11 – Svuota Lista:** azzera il contatore all'indirizzo 0 rendendo la lista vuota.

Il componente si interfaccia con la logica esterna attraverso un protocollo di handshake i_start/o_done e con una RAM sincrona di tipo write-first fornita nel testbench.

2 Architettura

L'architettura è organizzata come una macchina a stati finiti (FSM) con datapath.

2.1 Porte di Interfaccia

La Tabella 1 riassume i segnali di I/O del componente.

Segnale	Dir.	Bit	Descrizione
i_clk	IN	1	Clock di sistema (campionamento sul fronte di SALITA).
i_rst	IN	1	Reset ASINCRONO attivo alto. Forza la FSM in S_RESET.
i_start	IN	1	Handshake: portato a 1 per avviare un'operazione.
i_task_id	IN	6	ID del task da inserire (usato da OP10). Valori 1..63.
i_task_priority	IN	2	Priorità del task da inserire (00=max, 11=min).
i_op	IN	2	Selezione operazione: 00/01/10/11.
o_done	OUT	1	Handshake: uscita COMBINATORIA da S_DONE.
o_task_id	OUT	6	ID task estratto (OP01). 0 se lista vuota.
o_mem_addr	OUT	16	Indirizzo RAM (range 0..num_tasks).
i_mem_data	IN	8	Dato letto dalla RAM (valido dopo 1 ciclo).
o_mem_data	OUT	8	Dato da scrivere in RAM.
o_mem_we	OUT	1	Write Enable: 1=scrittura, 0=lettura.
o_mem_en	OUT	1	Enable memoria: 1 per ogni accesso.

Tabella 1: Segnali di interfaccia del modulo.

2.2 Segnali Interni — Registri (Flip-Flop)

Ogni segnale interno ha una versione current_* (valore attuale) e next_* (calcolato dalla logica combinatoria).

Segnale	Bit	Ruolo e motivazione della scelta
<code>current_state</code>	enum	Stato corrente della FSM. Registro centrale di controllo.
<code>num_tasks</code>	8	Copia locale di RAM[0]. Evita riletture continue della RAM.
<code>current_addr</code>	16	Indirizzo corrente per scansioni (OP00, OP01 shift, OP10).
<code>target_addr</code>	16	Posizione di inserimento (OP10). Salvata per l'uso post-shift.
<code>extracted_id</code>	6	ID estratto (OP01). Salvato perché <code>i_mem_data</code> viene sovrascritto.

Tabella 2: Registri interni del datapath.

2.3 Moduli e processi

2.3.1 Processo `state_reg`

Implementa il registro di stato con reset asincrono attivo alto: se `i_rst='1'` la FSM viene forzata in `S_RESET`. Sul fronte di salita del clock, `current_state` assume il valore di `next_state`.

2.3.2 Processo `datapath_regs`

Aggiorna i registri interni (`num_tasks`, `current_addr`, ecc.) sul fronte di salita del clock. Tale processo è utilizzato puramente per comodità di lettura e implementazione.

2.3.3 Processo `comb`

Processo combinatorio che determina le uscite e lo stato successivo. Include una sensitivity list completa come dettagliato nella Tabella 3.

Segnale	Perché è necessario nella sensitivity list
<code>current_state</code>	È il selettore del case: ogni cambio stato deve rieseguire la logica.
<code>i_start</code> , <code>i_op</code>	Ingressi primari letti in <code>S_IDLE</code> e <code>S_DECODE</code> .
<code>i_task_id</code> , ...	Usati per costruire il byte da scrivere o confrontare priorità.
Registri interni	Condizioni di terminazione, indirizzi e confronti.
<code>i_mem_data</code>	Dato letto dalla RAM, usato in quasi tutte le fasi operative.

Tabella 3: Dettaglio della Sensitivity List del processo `comb`.

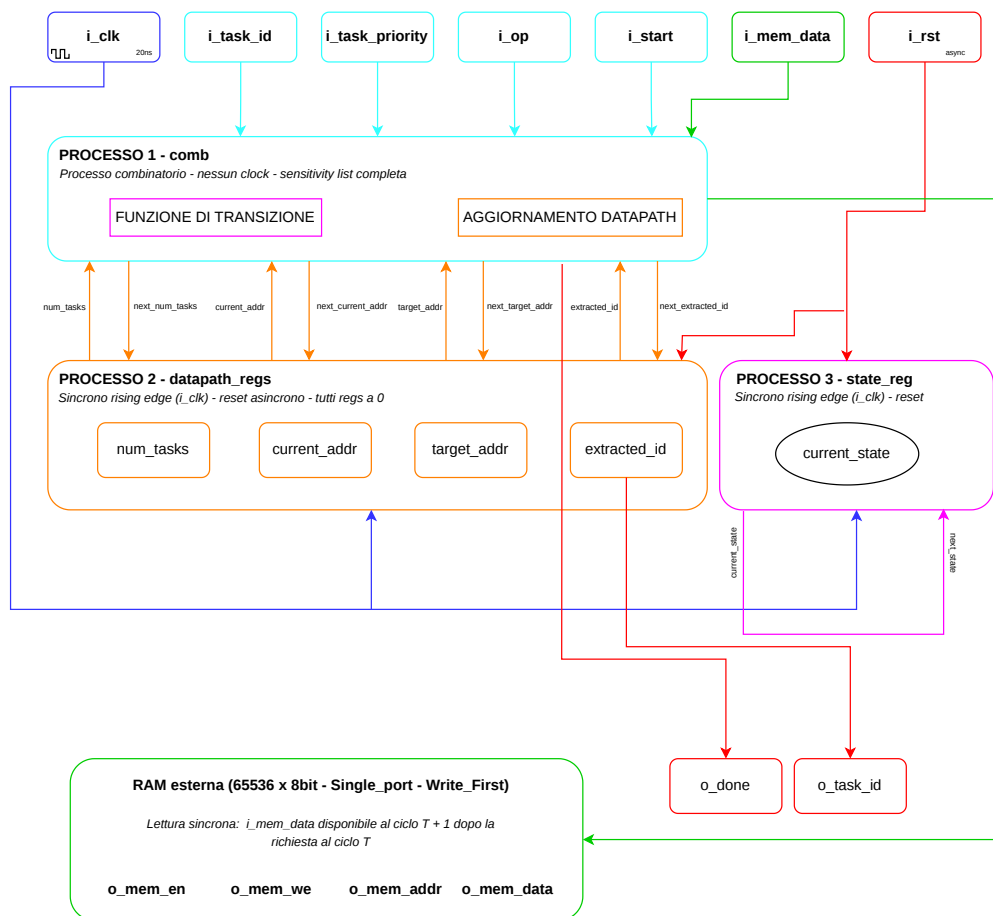


Figura 1: Schema a blocchi

2.4 Diagramma degli Stati

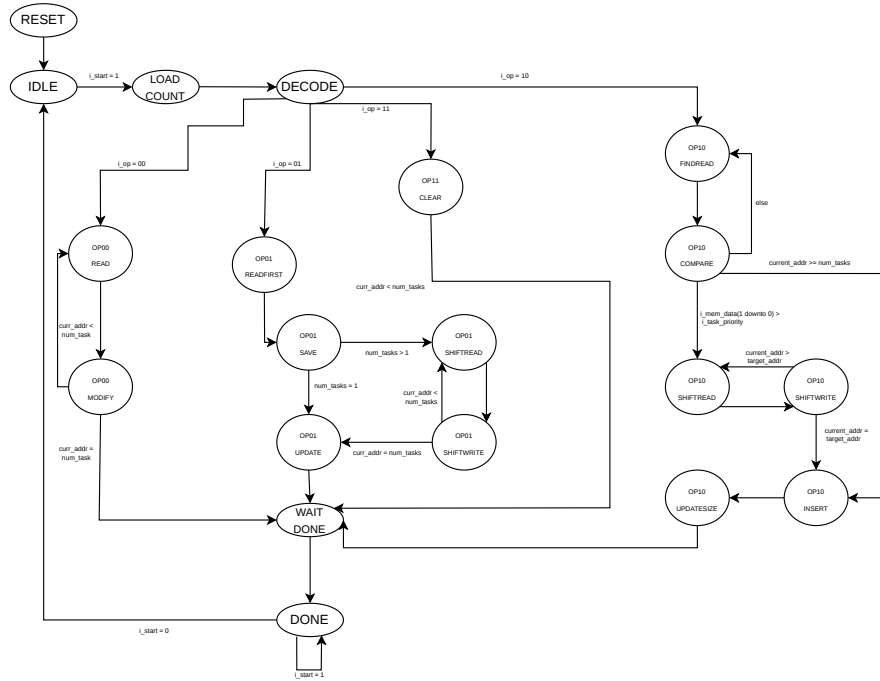


Figura 2: Schema dettagliato della FSMD.

Stato	Condizione di uscita	Stato successivo
S_RESET	-	S_IDLE
S_IDLE	i_start = '1'	S_LOAD_COUNT
S_LOAD_COUNT	-	S_DECODE
S_DECODE	i_op	S_OP00_READ / S_OP01_READ_FIRST / ...
S_OP00_READ	-	S_OP00_MODIFY
S_OP00_MODIFY	curr_addr < num_tasks	S_OP00_READ
S_OP00_MODIFY	curr_addr = num_tasks	S_WAIT_DONE
S_OP01_READ_FIRST	-	S_OP01_SAVE
S_OP01_SAVE	num_tasks = 1	S_OP01_UPDATE
S_OP01_SAVE	num_tasks > 1	S_OP01_SHIFT_READ
S_OP01_SHIFT_READ	-	S_OP01_SHIFT_WRITE
S_OP01_SHIFT_WRITE	current_addr = num_tasks	S_OP01_UPDATE
S_OP01_SHIFT_WRITE	current_addr < num_tasks	S_OP01_SHIFT_READ
S_OP01_UPDATE	-	S_WAIT_DONE
S_OP10_FIND_READ	-	S_OP10_COMPARE
S_OP10_COMPARE	i_mem_data(1 downto 0) > i_task_priority	S_OP10_SHIFT_READ
S_OP10_COMPARE	curr_addr >= num_tasks	S_OP10_INSERT
S_OP10_COMPARE	else	S_OP10_FIND_READ
S_OP10_SHIFT_READ	-	S_OP10_SHIFT_WRITE
S_OP10_SHIFT_WRITE	current_addr = target_addr	S_OP10_INSERT
S_OP10_SHIFT_WRITE	current_addr > target_addr	S_OP10_SHIFT_READ
S_OP10_INSERT	-	S_OP10_UPDATE_SIZE
S_OP10_UPDATE_SIZE	-	S_WAIT_DONE
S_OP11_CLEAR	-	S_WAIT_DONE
S_WAIT_DONE	-	S_DONE
S_DONE	i_start = '0'	S_IDLE

Tabella 4: Tabella delle transizioni della FSM.

3 Scelte progettuali

3.1 Gestione della RAM sincrona write-first

La RAM è sincrona con una latenza di un ciclo in lettura. Utilizziamo un pattern di accesso a due cicli (lettura/elaborazione) sfruttando il fatto che il dato è stabile nel ciclo $T+1$.

3.2 Controllo della lista vuota in S_DECODE

Lo stato S_DECODE verifica se la lista è vuota per diramare il flusso verso stati speciali, risparmiando cicli di clock. Inizialmente avevamo deciso di utilizzare degli stati di controllo (S_OPXX_CHEK_EMPTY) così da verificare dopo la scelta dell'operazione se la lista fosse vuota per poi saltare direttamente in uno stato terminale. Tale implementazione non era ottimale e assolutamente superflua.

3.3 Utilizzo dello stato S_WAIT_DONE

Un dettaglio tecnico che abbiamo considerato è lo stato S_WAIT_DONE. Nelle simulazioni iniziali, abbiamo notato che alzando o_done immediatamente dopo l'ultima scrittura, il Testbench poteva campionare la memoria prima che il fronte di clock avesse effettivamente consolidato il dato nella RAM sincrona. L'introduzione di un ciclo di latenza intenzionale tra l'ultima operazione di scrittura e l'attivazione di o_done garantisce una robustezza totale del sistema, assicurando che qualunque logica esterna legga dati perfettamente stabili e aggiornati.

3.4 Implementazione operazioni i_op

- **i_op = 00 (Decremento Priorità):** Il modulo esegue una scansione sequenziale dal primo all'ultimo task valido. Per ogni task, viene richiesta una lettura (S_OP00_READ) e, nel ciclo successivo (S_OP00_MODIFY), si incrementa il valore dei 2 bit di priorità se inferiori a "11". Il byte modificato viene riscritto immediatamente, saturando il valore a 3 per evitare overflow logici della priorità.
- **i_op = 01 (Estrazione Primo Task):** Il modulo accede all'indirizzo 1 per salvare l'ID del task da estrarre. Se la lista contiene più elementi, la FSM avvia un ciclo di shift a sinistra ($\text{addr}[i] \rightarrow \text{addr}[i-1]$), partendo dall'indice 2. Infine, il registro `num_tasks` viene decrementato e aggiornato in RAM. In caso di lista vuota, l'operazione termina restituendo ID 0.
- **i_op = 10 (Inserimento Task):** L'operazione è divisa in tre fasi: 1. *Ricerca*: Scansione della lista per trovare la posizione di inserimento corretta (politica FIFO: il nuovo task va dopo i pari priorità). 2. *Shift a destra*: Spostamento dei task esistenti ($\text{addr}[i] \rightarrow \text{addr}[i+1]$) partendo dal fondo per liberare la cella `target_addr`. 3. *Scrittura*: Inserimento del nuovo task e incremento del contatore all'indirizzo 0.
- **i_op = 11 (Svuota Lista):** L'operazione è implementata in modo atomico tramite la scrittura del valore zero all'indirizzo 0 della RAM. Poiché la validità dei dati dipende dal contatore di testa, non è necessario cancellare fisicamente i restanti task, portando l'esecuzione a un singolo ciclo di scrittura.

4 Risultati sperimentali

4.1 Simulazione Funzionale

La correttezza logica del design è stata verificata tramite *Behavioral Simulation* utilizzando il Test Bench fornito.

- **Origine dei dati:** I dati sono stati estratti dal log della *Tcl Console* e dalla finestra *Waveform Editor* al termine dell'esecuzione del comando `run all`.
- **Analisi del Log:** Il log di simulazione riporta il completamento con successo di tutti i 9 step di test previsti (Test step 0 a 8), terminando al tempo di **2 μ s** con il messaggio `Failure: Simulation Ended! TEST PASSATO`.
- **Validazione Temporale:** Come visibile dal grafico delle forme d'onda (Fig. 3), il segnale `tb_done` viene asserito correttamente in corrispondenza del termine dell'elaborazione, rispettando il periodo di clock impostato di **20.000 ps** (50 MHz).

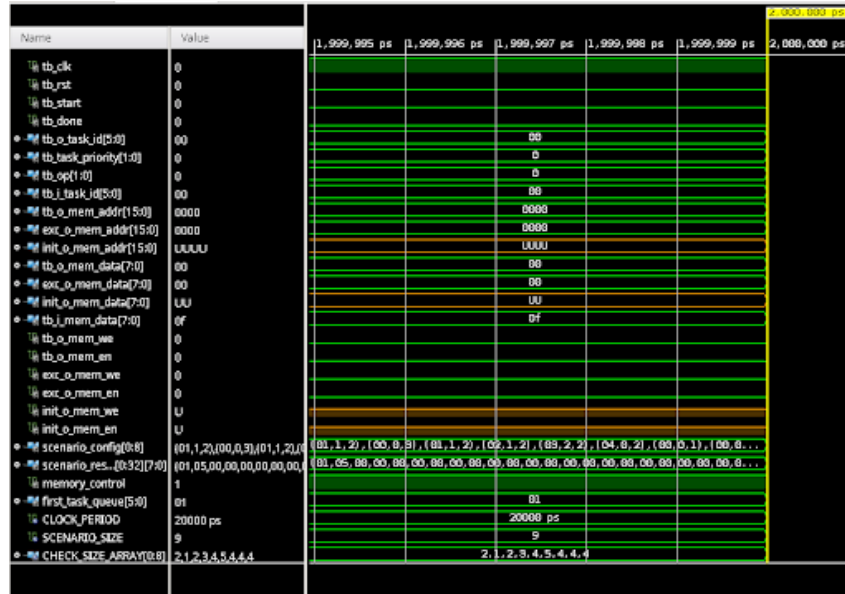


Figura 3: Waveforms

4.2 Sintesi e Occupazione di Area

Dopo aver eseguito il processo di *Run Synthesis*, è stato generato il *Report Utilization* per valutare l'impatto del design sulle risorse della FPGA target (Xilinx Artix-7).

- **Origine dei dati:** Tabella estratta dal report testuale generato dal comando `report_utilization`.
- **Risorse Logiche:** Il design utilizza **210 Slice LUTs** (0.51% del totale disponibile), indicando un'implementazione della logica combinatoria estremamente efficiente.
- **Elementi di Memoria:** Sono stati impiegati **66 Slice Registers** (0.08%), che corrispondono ai registri definiti nel datapath per la memorizzazione di indirizzi e dati temporanei.
- **Qualità del Design:** Il report conferma la presenza di **0 Register as Latch**, validando la natura puramente sincrona del sistema.

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	210	0	41000	0.51
LUT as Logic	210	0	41000	0.51
LUT as Memory	0	0	13400	0.00
Slice Registers	66	0	82000	0.08
Register as Flip Flop	66	0	82000	0.08
Register as Latch	0	0	82000	0.00
F7 Muxes	0	0	20500	0.00
F8 Muxes	0	0	10250	0.00

Figura 4: Dettaglio del report di utilizzazione delle risorse.

4.3 Analisi Temporale (Timing)

L'analisi dei ritardi è stata effettuata tramite il *Report Timing Summary* post-sintesi.

- **Origine dei dati:** Analisi del *Worst Case Path* estratta tramite il comando `report_timing_summary`.
- **Percorso Critico:** Il ritardo massimo registrato (*Data Path Delay*) è di **5.424 ns**. Il percorso critico ha origine dal registro `current_addr_reg[2]/C` e termina sulla porta di uscita `o_mem_addr[14]`.
- **Composizione del Ritardo:** Il ritardo è composto per il **69.22%** da logica (livelli di *CARRY4* e *LUT*) e per il **30.8%** da routing.
- **Considerazioni sulla frequenza:** Il design presenta uno *Slack* infinito (o ampiamente positivo rispetto ai 20ns di periodo), garantendo che il sistema possa operare senza violazioni di setup a 50 MHz.

Listing 1: Dettaglio del percorso critico (Critical Path Analysis)

```

1 Slack: inf
2 Source: current_addr_reg[2]/C
3           (rising edge-triggered cell FDCE)
4 Destination: o_mem_addr[14]
5           (output port)
6
7 Path Type: Max at Slow Process Corner
8 Data Path Delay: 5.424ns (logic 3.755ns (69.2%) route 1.670ns (30.8%))
9 Logic Levels: 9 (CARRY4=4 FDCE=1 LUT1=1 LUT3=1 LUT6=1 OBUF=1)
10
11   Location Delay type Incr(ns) Path(ns)
12   -----
13           FDCE 0.000 0.000 r
14 current_addr_reg[2]/Q (Prop_fdce_C_Q) 0.269 0.269 r
15 ...
16 CARRY4 (Prop_carry4_CI_C0[3]) 0.058 1.145 r
17 ...
18 o_mem_addr_OBUF[14] (Prop_obuf_I_0) 2.462 5.424 r
19   -----

```

Legenda dei dati del Timing Report:

FDCE (Source): Punto di innesco del percorso critico; il ritardo include il *Clock-to-Q* del registro `current_addr_reg[2]`.

CARRY4: Celle di logica veloce per la propagazione del riporto, utilizzate qui per l'incremento/calcolo degli indirizzi di memoria.

LUT1/3/6: Unità logiche combinatorie (Look-Up Tables) che processano i segnali di controllo e i dati degli stati della FSM.

OBUF (Destination): Buffer di uscita verso il pin fisico `o_mem_addr[14]`. Rappresenta il ritardo di "uscita" dal chip, che influisce per circa il 45% sul ritardo totale.

Data Path Delay: Tempo totale di volo del segnale dal registro interno al pin di uscita (5.424 ns).

4.4 Tabella riassuntiva

Parametro	Valore Ottenuto	Origine del Dato
Esito Simulazione	TEST PASSATO	Log Tcl Console
Tempo di Esecuzione	2 μ s	Waveform Editor
Periodo di Clock	20.000 ps (50 MHz)	Testbench / Waveform Editor
Utilizzo LUT	210 (0.51%)	Report Utilization (Site Type)
Utilizzo Registri	66 (0.08%)	Report Utilization (Registers)
Register as Latch	0	Report Utilization (Registers)
Worst Case Delay	5.424 ns	Timing Summary (Critical Path)
Logic Levels	9	Timing Summary
Frequenza Max Stimata (F_{max})	$\approx$ 184.3 MHz	Calcolo su Data Path Delay

Tabella 5: Riepilogo delle metriche di sintesi e validazione temporale

4.5 Nota sui dati

I valori di utilizzazione e timing possono variare leggermente tra diverse iterazioni di sintesi a causa degli algoritmi euristici di Place and Route del tool Vivado, che cerca di ottimizzare il piazzamento dei componenti in base ai vincoli impostati. Ad esempio, con una ulteriore sintesi abbiamo notato delle differenze in alcuni dati ottenuti, (ad esempio 195 LUT e 7.317 ns WCD, Logic Levels 4 ...). Questi valori sono normali, infatti si tratta di un tradeoff tra area e tempo.

- Meno LUT (**195**): Il sintetizzatore è riuscito a compattare la logica, magari usando meno componenti ma creando catene logiche più lunghe.
- Più ritardo (**7.317 ns**): Avendo compattato la logica o avendo scelto un posizionamento dei componenti meno "diretto", il segnale impiega più tempo ad attraversare il percorso critico. Rimane comunque un risultato "safe", infatti la frequenza massima è $\approx$ 136MHz che è molto maggiore di 50MHz.

5 Testbench Ad Hoc

5.1 Introduzione alla Strategia di Test

La strategia di testing che abbiamo adottato si è basata sull'architettura del testbench fornito nelle specifiche di input, espandendone le funzionalità per testare gli edge cases.

Per fare ciò, sono stati sviluppati diversi file di testbench derivati dal modello originale. Ciascun test è stato strutturato per sollecitare specifiche transizioni della macchina a stati e verificare la corretta manipolazione della memoria RAM, assicurando che il sistema mantenga la coerenza dei dati anche a seguito di sequenze di operazioni insolite o segnali di controllo asincroni.

5.2 Descrizione dei Testbench per Casi Limite

Di seguito vengono analizzati nel dettaglio gli scenari testati per verificare la robustezza del modulo.

TB_EDGE_01 (Rimozione da lista vuota): Verifica il comportamento dell'operazione di estrazione (OP=01) quando il contatore task in RAM[0] è nullo. Il sistema deve ignorare la richiesta, restituendo un o_task_id nullo (000000) e lasciando la memoria invariata.

TB_EDGE_02 (Decremento priorità su lista vuota): Analizza l'operazione di decremento (OP=00) in assenza di task. Assicura che la logica di scansione non si attivi erroneamente, prevenendo accessi a locazioni di memoria non valide.

- TB_EDGE_03 (Saturazione priorità):** Verifica il meccanismo di saturazione superiore. Un task con priorità massima (11 binario) soggetto a decremento deve mantenere il valore senza causare overflow logici.
- TB_EDGE_04 (Saturazione mista):** Scenario di stress-test che alterna inserimenti e decrementi multipli, verificando che la saturazione delle priorità operi correttamente su diversi task durante la stessa sequenza operativa.
- TB_EDGE_05 (Gestione parità di priorità):** Valida la politica FIFO tra task con la stessa priorità. Il sistema deve inserire il nuovo task subito dopo quelli esistenti aventi medesima priorità.
- TB_EDGE_06 (Reset durante il lavoro):** Verifica che il sistema torni correttamente allo stato iniziale se riceve un segnale di reset proprio mentre sta scrivendo in memoria o spostando i dati. Questo garantisce che il modulo non rimanga bloccato in uno stato inconsistente se viene interrotto bruscamente durante un'operazione.
- TB_EDGE_07 (Doppio reset consecutivo):** Valida la stabilità del segnale di reset e la sincronizzazione del segnale DONE, assicurando che impulsi consecutivi non inducano stati di stallo o errori di protocollo.

6 Conclusioni

6.1 Considerazioni finali sul lavoro svolto

L'approccio che abbiamo adottato è stato caratterizzato da una forte attenzione alla pianificazione dell'architettura: piuttosto che procedere per tentativi, abbiamo investito tempo nella definizione di una macchina a stati capace di gestire la logica generale.

Proprio durante la stesura del codice, però, abbiamo riscontrato alcune criticità che ci hanno portato a rivedere l'architettura iniziale. Inizialmente, ad esempio, avevamo previsto degli stati di "check-empty" specifici per ogni singola operazione; tuttavia, ci siamo resi conto che questi rappresentavano una ridondanza. Abbiamo quindi ottimizzato il design spostando il controllo sulla validità della lista direttamente nello stato di DECODE. Questa scelta non solo ha permesso di ridurre il numero totale degli stati della FSM, ma ha anche migliorato sensibilmente la leggibilità del codice e la comprensione logica dell'intero sistema.

Al di là della codifica, è stato particolarmente stimolante dedicarsi all'analisi dei dati prodotti dai report e dalle simulazioni. È stato estremamente interessante osservare come il nostro codice si comportava "fisicamente" all'interno della FPGA sotto determinate condizioni. In particolare, abbiamo notato con interesse come iterazioni diverse della sintesi potessero produrre risultati leggermente differenti in termini di occupazione di area e timing. Osservare questi *trade-off* (dove a volte un risparmio nel numero di LUT comportava un leggero aumento nel ritardo del percorso critico) ci ha permesso di toccare con mano il compromesso tra area e tempo.

Particolarmente significativa è stata la fase di verifica. Basandoci sulla struttura del testbench fornito, abbiamo scelto di non limitarci a una validazione funzionale di base, ma di estendere il testing a una serie di scenari critici.